

ALGORITMI E STRUTTURE DATI

ANALISI DEGLI ALGORITMI

AULA VIRTUALE DEL 04-05-2026

ANALISI AMMORTIZZATA

In molte situazioni pratiche, ci troviamo ad eseguire un algoritmo ripetutamente nel tempo. Si pensi, ad esempio, ai motori di ricerca su Internet vedono arrivare miliardi di interrogazioni al giorno, ognuna delle quali corrisponde all'esecuzione di un certo algoritmo di ricerca. In contesti di questo tipo che abbiamo a che fare con collezioni di dati soggetti a sequenze di interrogazioni ed aggiornamenti.

ANALISI AMMORTIZZATA

Useremo il termine **operazione di aggiornamento** per indicare una operazione che modifica il contenuto informativo di una collezione di dati. Viceversa parleremo di **operazione di interrogazione** per denotare una operazione che esplora i dati di una collezione senza modificarne il contenuto informativo.

L'**analisi ammortizzata** studia le prestazioni **medie** di una sequenza di operazioni su una collezione di dati, piuttosto che della singola esecuzione di un algoritmo come abbiamo visto finora.

ANALISI AMMORTIZZATA

Studiare il comportamento di un algoritmo su una sequenza di esecuzioni può darne una caratterizzazione più raffinata, in sostanza nell'analisi ammortizzata le probabilità non entrano in gioco in nessun modo.

Ad esempio, una singola esecuzione di un algoritmo potrebbe richiedere tempo $\Theta(n)$ nel caso peggiore, mentre n esecuzioni potrebbero richiedere tempo totale ancora $\Theta(n)$ nel caso peggiore, e non $\Theta(n^2)$ come potremmo pensare.

ANALISI AMMORTIZZATA

Una singola esecuzione potrebbe richiedere molto tempo per via di una configurazione particolarmente svantaggiosa dei dati su cui opera. Tuttavia, i dati stessi potrebbero configurarsi in modo molto più vantaggioso durante le esecuzioni successive, per cui in media il tempo richiesto per ciascuna esecuzione potrebbe essere di fatto molto più basso di quello che si avrebbe nel singolo caso peggiore. Per caratterizzare le prestazioni di un algoritmo in media su una sequenza di operazioni, parleremo del concetto di **costo ammortizzato**.

ANALISI AMMORTIZZATA

Def. Costo Ammortizzato: Definiamo il costo ammortizzato di un algoritmo su una sequenza di k operazioni come:

$$T_a(n) = \frac{T(n,k)}{k}$$

dove T(n,k) è il tempo totale richiesto dall'algoritmo nel caso peggiore per tutte le k operazioni su istanza di dimensione n.

ANALISI AMMORTIZZATA

Vi sono diversi metodi per calcolare il costo ammortizzato di un’operazione: fra quelli più noti, il metodo dei crediti e il metodo del potenziale. La scelta di un metodo piuttosto che un altro è una questione di convenienza rispetto al particolare algoritmo che si sta considerando.

ANALISI AMMORTIZZATA: IL METODO DEI CREDITI

In alcuni casi può accadere che il costo di un'operazione dipenda strettamente dall'esecuzione nel passato di altre operazioni: per esempio una delete può rimuovere elementi precedentemente inseriti da altrante insert. Il metodo dei crediti permette di calcolare il costo ammortizzato di una operazione senza dover entrare nel dettaglio di queste dipendenze, che possono essere, a volte, difficili da determinare con precisione. L'idea è quella di associare delle monete immaginarie (o crediti) agli oggetti di una collezione. Queste monete, depositate da alcune operazioni, saranno in grado di «pagare» il costo di altre operazioni.

ANALISI AMMORTIZZATA: IL METODO DEI CREDITI

Assumeremo che:

1 moneta vale O(1) passi di calcolo

«come diceva Paperon de Paperoni «il tempo è denaro».

Chiaramente l’uso delle monete serve solo per l’analisi e non per il funzionamento.

ANALISI AMMORTIZZATA: IL METODO DEI CREDITI

Def. Metodo dei crediti: Il costo ammortizzato di una operazione secondo il metodo dei crediti è definito come:

$$T_a(n) = T(n) + deposito(n) - prelievo(n),$$

Dove T(n) è il numero di passi effettivo che l'operazione compie nel caso peggiore, deposito(n) è il numero di monete che deposita sugli oggetti, e prelievo(n) è il numero di quelle che preleva dagli oggetti.

ANALISI AMMORTIZZATA: IL METODO DEI CREDITI

In generale, assoceremo agli oggetti zero o più monete in base a qualche loro proprietà, e poi definiremo prelievo e deposito in modo che questa associazione venga rispettata tenendo conto di come gli oggetti stessi sono manipolati dalle operazioni.

ANALISI AMMORTIZZATA: IL METODO DEL POTENZIALE

Il metodo del potenziale è concettualmente simile al metodo dei crediti. La differenza principale è che, invece di associare crediti ai singoli oggetti, associamo piuttosto un potenziale ai dati nel loro insieme (in sostanza un numero non negativo). L'associazione avviene tramite una funzione potenziale $\Phi(I)$ che applicata ad una istanza I della struttura dati, definisce il potenziale dell'istanza.

ANALISI AMMORTIZZATA: IL METODO DEL POTENZIALE

Analogamente ai crediti, il potenziale di una collezione di oggetti può servire a pagare il costo di certe operazioni sulla collezione stessa. Il concetto è del tutto analogo a quello di un corpo in un campo elettrico o gravitazionale a cui è associata una **energia potenziale** che può essere trasformata in lavoro.

ANALISI AMMORTIZZATA: IL METODO DEL POTENZIALE

Def. Metodo del potenziale: Il costo ammortizzato di una operazione secondo il metodo del potenziale è definito come:

$$T_a(n) = T(n) + \Phi(I') - \Phi(I),$$

Dove T(n) è il numero di passi effettivo che l'operazione compie nel caso peggiore, I è l'istanza di dimensione n della struttura dati in ingresso all'operazione ed I' è l'istanza della struttura dati dopo l'esecuzione dell'operazione.

ANALISI AMMORTIZZATA: IL METODO DEL POTENZIALE

Mostriamo ora che sotto opportune condizioni, la somma dei costi ammortizzati secondo il metodo del potenziale fornisce una delimitazione superiore al tempo totale richiesto nel caso peggiore dell'intera sequenza:

Teorema. Siamo $I_0, \ldots, I_k$ le istanze di dimensione n della struttura dati ottenute su una sequenza di k operazioni a partire da un'istanza iniziale I_0. Se $\Phi(I_0) = 0$ e $\Phi(I_k) >= 0$, allora

T(n,k) <= $\sum_{i=1}^{k} T_a(n)$, dove T(n,k) è il tempo totale richiesto dall'algoritmo nel caso peggiore per tutte le k operazioni su istanze di dimensione n.

MODELLI AVANZATI: LA GERARCHIA DI MEMORIA

Per completezza, accenniamo ad un modello più sofisticato rispetto ai modelli presentati finora ma che meglio si presta ad astrarre una importante caratteristica delle moderne piattaforme di calcolo. Si parla infatti di gerarchia di livelli di memoria, ciascuno con un proprio costo e le proprie caratteristiche prestazionali.

E' quindi importate che i dati importati tramite un singolo accesso a disco, fintantoché si trovano in memoria centrale, siano «sfruttati» il più possibile dall'algoritmo.

MODELLI AVANZATI: LA GERARCHIA DI MEMORIA

In altre parole, se un algoritmo esibisce proprietà di località nell'accesso ai dati, il numero di I/O viene minimizzato, ottenendo prestazioni migliori. Per analizzare la caratteristica di località di accesso ai dati di un algoritmo, la classica macchina a registri è stata estesa in modo da supportare una visione a due livelli della memoria ed essere resa più realistica. Il modello di memoria esterna cattura quanto detto nel modo seguente:

MODELLI AVANZATI: LA GERARCHIA DI MEMORIA

- La memoria esterna ha una dimensione illimitata;
- La memoria interna ha una dimensione finita M;
- I dati sono trasferiti dalla memoria esterna a quella interna (e viceversa) in blocchi di dimensione B. Ossia si assume che $B < M/2$.

ANALISI DELLO SPAZIO (USO DELLA MEMORIA)

L’analisi dello spazio misura quanta memoria usa un algoritmo in funzione della dimensione dell’input n.

Due componenti principali

- Spazio input: memoria necessaria per memorizzare i dati (spesso non si conta)
- Spazio ausiliario: memoria extra usata dall’algoritmo

ANALISI DELLO SPAZIO (USO DELLA MEMORIA)

Algoritmo in-place

- Usa solo poche variabili
- Spazio: O(1)

```
def reverse(A):
    i, j = 0, len(A)-1
    while i < j:
        A[i], A[j] = A[j], A[i]
        i += 1
        j -= 1
```

ANALISI DELLO SPAZIO (USO DELLA MEMORIA)

Algoritmo con spazio extra

- Usa un altro array
- Spazio: O(n)

```
def copia(A):
    B = [0]*len(A)
    for i in range(len(A)):
        B[i] = A[i]
```

ANALISI DELLO SPAZIO (USO DELLA MEMORIA)

Attenzione alla ricorsione

Ogni chiamata ricorsiva usa stack:

- Fibonacci ricorsivo → O(n) spazio
- Merge sort → O(n) spazio

Punti di attenzione:

- Ottimizzare spazio può rendere l'algoritmo più efficiente in pratica
- A volte si scambia spazio per tempo (time–space tradeoff)

STABILITA' DEGLI ALGORITMI

Un algoritmo (di solito di ordinamento) è stabile se: mantiene l'ordine relativo degli elementi uguali. Esempio:
Input (valore, etichetta):
(5,A), (3,B), (5,C)
Dopo ordinamento per valore:

- Stabile (3,B), (5,A), (5,C)
- Non stabile (3,B), (5,C), (5,A)

Perché è importante? Quando gli elementi hanno più chiavi. Es: ordino prima per nome, poi per età. La stabilità è fondamentale quando i dati hanno informazioni implicite nell'ordine

CONFRONTO TRA EFFICIENZA DEGLI ALGORITMI

Serve per capire quale algoritmo è migliore per un certo problema.
Criteri principali

- Tempo (time complexity)O(1), O(log n), O(n), O(n log n), $O(n^2)$
- Spazio (space complexity)O(1), O(n), …

Algoritmo	Tempo medio	Spazio	Stabile
Bubble Sort	$O(n^2)$	O(1)	✔
Merge Sort	O(n log n)	O(n)	✔
Quick Sort	O(n log n)	O(log n)	✘

CONFRONTO TRA EFFICIENZA DEGLI ALGORITMI

Trade-off tipici

- Merge Sort:
  - veloce
  - usa memoria
- Quick Sort:
  - veloce e poco spazio
  - non stabile

CONFRONTO TRA EFFICIENZA DEGLI ALGORITMI

Non esiste "l'algoritmo migliore in assoluto"

Dipende da:

- dimensione input
- memoria disponibile
- necessità di stabilità
- caso medio vs peggiore

ALGORITMI DI ORDINAMENTO: BUBBLE SORT

L'idea di base è far "venire a galla" (come bolle) gli elementi più grandi verso la fine dell'array.

- **Come funziona:** L'algoritmo scorre l'array più volte. Ad ogni passaggio, confronta coppie di elementi adiacenti. Se sono nell'ordine sbagliato (il primo è maggiore del secondo), li scambia. Dopo la prima passata, l'elemento più grande si troverà sicuramente nell'ultima posizione. Il processo si ripete, ignorando via via gli ultimi elementi che sono già al posto giusto.
- **Complessità Temporale:** O(n^2) nel caso pessimo e medio. Se implementato con una flag che rileva se sono avvenuti scambi in una passata (ottimizzazione), il caso ottimo (array già ordinato) scende a O(n).

ALGORITMI DI ORDINAMENTO: BUBBLE SORT

- **Caratteristiche:** Fa moltissimi scambi ed è un algoritmo **stabile** (non altera l'ordine relativo di elementi con lo stesso valore).
- **Uso pratico:** Quasi nullo, serve principalmente a scopo didattico.

ALGORITMI DI ORDINAMENTO: SELECTION SORT

L'idea qui è dividere logicamente l'array in due parti: una parte ordinata (all'inizio) e una parte da ordinare (alla fine).

- **Come funziona:** Cerca l'elemento più piccolo in tutto l'array non ordinato. Una volta trovato, lo scambia con il primo elemento della parte non ordinata. Ora la parte ordinata è cresciuta di un elemento. Ripete il processo finché l'array non è completamente ordinato.
- **Complessità Temporale:** O(n^2) sempre: nel caso pessimo, medio e ottimo. Deve sempre scansionare tutta la parte rimanente dell'array per trovare il minimo, anche se l'array è già ordinato.

ALGORITMI DI ORDINAMENTO: SELECTION SORT

- **Caratteristiche:** Il suo grande vantaggio è che fa il **minor numero possibile di scambi** (al massimo O(n)). È utile se il costo di scrittura in memoria è estremamente elevato rispetto al costo di lettura (situazione rara oggi).
- Generalmente **non è stabile**.
- **Uso pratico:** Situazioni in cui la memoria in scrittura è un collo di bottiglia estremo

ALGORITMI DI ORDINAMENTO: INSERTION SORT

Ordinamento per Inserzione ed è il modo in cui la maggior parte delle persone ordina le carte da gioco in mano. Anche qui dividiamo l'array in parte ordinata e parte da ordinare.

- **Come funziona:** Prende il primo elemento della parte non ordinata e lo "inserisce" nella posizione corretta all'interno della parte già ordinata, facendogli spazio spostando gli elementi più grandi verso destra.
- **Complessità Temporale:** O(n^2) nel caso pessimo e medio (es. array ordinato al contrario). Tuttavia, il caso ottimo (array già ordinato) è **O(n)**. Inoltre, è estremamente efficiente su array quasi ordinati o array molto piccoli.

ALGORITMI DI ORDINAMENTO: INSERTION SORT

- **Caratteristiche:** Fa molti spostamenti (shift) ma meno confronti medi rispetto al Bubble Sort. È un algoritmo **stabile**.
- **Uso pratico:** Altissimo. Paradossalmente, proprio per la sua efficienza su array minuscoli (solitamente meno di 10-20 elementi), algoritmi avanzati come il Quick Sort o il Merge Sort passano all'Insertion Sort quando i sotto-problemi diventano sufficientemente piccoli. (Questo algoritmo ibrido si chiama *TimSort* ed è l'algoritmo di ordinamento di default in Python e Java per gli oggetti).

ALGORITMI DI ORDINAMENTO: QUICK SORT

È l'algoritmo di ordinamento più utilizzato in pratica (spesso integrato nelle librerie standard di molti linguaggi) perché, nonostante il suo caso pessimo sia teoricamente O(n^2), nel caso medio e pratico le costanti nascoste sono bassissime, rendendolo estremamente veloce. Opera in loco (non richiede array extra, a parte lo stack delle chiamate ricorsive), ma non è stabile.

- Scelta del pivot: È il cuore del Quick Sort. Il pivot è un elemento scelto dall'array che farà da "spartiacque".
  - Se scegli sempre il primo o l'ultimo elemento e l'array è già ordinato, finisci nel caso pessimo O(n^2).

ALGORITMI DI ORDINAMENTO: QUICK SORT

- Le strategie moderne prevedono la scelta di un pivot casuale (Randomized Quick Sort) o la tecnica "Mediana di tre" (si guarda il primo, il centrale e l'ultimo elemento, e si sceglie il valore mediano). Una buona scelta del pivot garantisce che l'array venga diviso in due metà quasi uguali, mantenendo la complessità a O(n log n).

- Partizionamento: Una volta scelto il pivot, l'algoritmo riorganizza l'array in modo che tutti gli elementi minori del pivot finiscano alla sua sinistra, e tutti i maggiori alla sua destra. Alla fine di questa fase, il pivot si trova nella sua posizione finale definitiva. A questo punto, l'algoritmo richiama se stesso ricorsivamente sulla sotto-metà sinistra e sulla sotto-metà destra.

ALGORITMI DI ORDINAMENTO: MERGE SORT

È l'algoritmo "Divide et Impera" per eccellenza. Il suo pregio più grande è l'assoluta prevedibilità: costa **sempre O(n log n)**, sia nel caso pessimo, medio che ottimo. Inoltre, è un algoritmo **stabile**.
Il suo svantaggio principale è che **non opera in loco**: richiede una memoria ausiliaria pari a O(n) per unire i sotto-array.

- **Fase di divisione:** È la parte banale. L'array viene diviso a metà, poi le metà vengono divise a metà, e così via in modo ricorsivo finché non si ottengono n sotto-array di dimensione 1. (Un array di 1 elemento è, per definizione, ordinato).

ALGORITMI DI ORDINAMENTO: MERGE SORT

- **Fase di merge (fusione):** È qui che avviene il vero lavoro. L'algoritmo prende due sotto-array già ordinati e li fonde in un unico array ordinato più grande. Per farlo, usa due "puntatori" che scorrono i due sotto-array: confronta gli elementi puntati, prende il più piccolo, lo copia nell'array ausiliario e fa avanzare il puntatore corrispondente. Questa fusione a cascata continua finché l'intero array non è ricomposto.

ALGORITMI DI ORDINAMENTO: HEAP SORT

L'Heap Sort abbandona il "Divide et Impera" puro e si affida all'efficienza di una struttura dati ad albero chiamata **Heap** (precisamente un Max-Heap per ordinare in senso crescente). Un Max-Heap è un albero binario quasi completo in cui il valore di ogni nodo padre è maggiore o uguale a quello dei suoi figli. L'Heap Sort garantisce un tempo O(n log n) nel caso pessimo e opera **in loco**, ma **non è stabile**.

- **Costruzione Heap (Build-Max-Heap):** L'algoritmo prende l'array disordinato e lo riorganizza logicamente per rispettare le proprietà del Max-Heap. Sorprendentemente, questa operazione iniziale si può fare in tempo O(n) partendo dal basso verso l'alto (eseguendo un'operazione chiamata *heapify* sui nodi interni).

ALGORITMI DI ORDINAMENTO: HEAP SORT

- **Fase di ordinamento:** Una volta costruito il Max-Heap, sappiamo per certo che l'elemento più grande in assoluto si trova nella radice (la prima posizione dell'array). L'algoritmo prende questo elemento massimo e lo scambia con l'ultimo elemento dell'array. Ora il numero più grande è al suo posto definitivo alla fine. L'albero rimanente (più piccolo di 1) viene "aggiustato" (*heapify down*) per far risalire il nuovo massimo alla radice. Si ripete finché l'albero non si svuota.

ALGORITMI DI ORDINAMENTO: CONFRONTO

Quick Sort vs Merge Sort

È il confronto più classico. Entrambi usano il *Divide et Impera*, ma con filosofie opposte.

- **Tempo:** Entrambi hanno caso medio O(n log n). Il Merge Sort lo garantisce anche nel caso pessimo, mentre il Quick Sort può degenerare a O(n^2) (se il pivot è scelto male).
- **Spazio:** Quick Sort è **in loco** (usa solo O(log n) per lo stack delle chiamate ricorsive). Merge Sort richiede un array ausiliario di dimensione O(n) per la fusione.
- **Stabilità:** Merge Sort è stabile, Quick Sort no.
- **Chi vince in pratica?** Il **Quick Sort**. Grazie al modo in cui analizza array contigui, ha una **località di riferimento (cache locality)** eccellente a livello di CPU, rendendo le sue costanti nascoste molto più piccole rispetto al Merge Sort.

ALGORITMI DI ORDINAMENTO: CONFRONTO

Quick Sort vs Bubble Sort

Non c'è vera partita, ma serve a capire l'evoluzione.

- Bubble Sort è O(n^2), Quick Sort è mediamente O(n log n). Su array di grandi dimensioni, il Quick Sort finisce in frazioni di secondo ciò che il Bubble Sort impiega ore a fare.
- L'unica magra consolazione del Bubble Sort è che è stabile e, se ottimizzato, impiega O(n) su un array già ordinato (mentre un Quick Sort ingenuo su un array ordinato cade nel suo caso pessimo O(n^2)).

ALGORITMI DI ORDINAMENTO: CONFRONTO

Heap Sort vs Merge Sort vs Quick Sort (I "Big Three")

- **Quick Sort:** Il re della pratica (veloce, in loco), ma instabile e rischioso nel caso pessimo.
- **Merge Sort:** L'affidabile (sempre O(n log n), stabile), ma "sprecone" in termini di memoria (O(n)).
- **Heap Sort:** Il compromesso teorico perfetto (sempre O(n log n) e in loco), ma **instabile e lento in pratica** rispetto al Quick Sort a causa dei continui salti nell'array (scarsa cache locality) per calcolare padri e figli.

ALGORITMI DI ORDINAMENTO: CONFRONTO

Insertion Sort vs Selection Sort

Entrambi O(n^2), ma si comportano diversamente.

- **Insertion Sort** è **adattivo**: se l'array è parzialmente ordinato, interrompe i confronti in anticipo. Selection Sort invece scansiona sempre tutto, costando O(n^2) anche se l'array è già perfetto.
- **Selection Sort** fa al massimo O(n) **scambi**. Insertion Sort fa moltissimi spostamenti in memoria. Se scrivere in memoria costa infinitamente più che leggere (es. memorie flash antiche), vince Selection Sort. In tutti gli altri casi, vince Insertion Sort.

ALGORITMI DI ORDINAMENTO: CONFRONTO

Insertion Sort vs Bubble Sort

Entrambi O(n^2), stabili e adattivi (O(n) nel caso ottimo).

- **Insertion Sort vince a mani basse.** Il Bubble Sort esegue uno scambio completo (3 operazioni di assegnamento) ad ogni passo. L'Insertion Sort fa solo un singolo "shift" (1 operazione di assegnamento) per spostare gli elementi verso destra.

ALGORITMI DI ORDINAMENTO: CONFRONTO

Insertion Sort vs Heap Sort

Perché confrontare un algoritmo O(n^2) con uno O(n log n)? Per una questione di **dimensione dell'input (overhead)**.

- Su n molto grande, Heap Sort vince clamorosamente.
- Su n molto piccolo (generalmente n < 16 o n < 20), **Insertion Sort è più veloce**. Non ha bisogno di creare l'albero logico, non ha chiamate ricorsive, è banalmente un ciclo while compatto che sta tutto nella cache del processore.

ALGORITMI DI ORDINAMENTO: OTTIMIZZAZIONI E VARIANTI

Un piccolo focus sull'Insertion Sort

Molti preferiscono l'Insertion Sort perché è il "migliore tra i peggiori" ed è il cuore di molte ottimizzazioni moderne.

Insertion Sort su Array Quasi Ordinati
Un array "quasi ordinato" è un array in cui ogni elemento si trova a non più di una distanza costante k dalla sua posizione finale corretta, oppure dove il numero di inversioni (coppie disordinate) è molto basso.

- In questo scenario, il ciclo interno dell'Insertion Sort si ferma quasi subito.
- La complessità crolla da O(n^2) a **O(n + d)**, dove d è il numero di inversioni. Se d è proporzionale a n, il tempo di esecuzione diventa **Lineare O(n)**.

ANALISI DEGLI ALGORITMI

Referenze

- Algoritmi e Strutture Dati – Demetrescu, Finocchi, Italiano
- Introduzione agli algoritmi e strutture dati – Cormen, Leiserson, Rivest, Stein

Risorse Python

https://runestone.academy/ns/books/published/pythonds/index.html?utm_source=chatgpt.com

https://github.com/0bprashanthc/algorithm-books/blob/master/Data%20Structures%20%26%20Algorithms%20in%20Python.pdf?utm_source=chatgpt.com

https://edu.anarcho-copy.org/Programming%20Languages/Python/Python%20Data%20Structures%20and%20Algorithms.pdf?utm_source=chatgpt.com

https://do1.dr-chuck.com/pythonlearn/IT_it/pythonlearn.pdf?utm_source=chatgpt.com

